

Notifications

Architecture, current implementation status, and what remains before FCM push goes live

Liaison Officer module · Prepared 27 September 2026 · Ref: requirement #11 / LO.9.4

1. At a glance

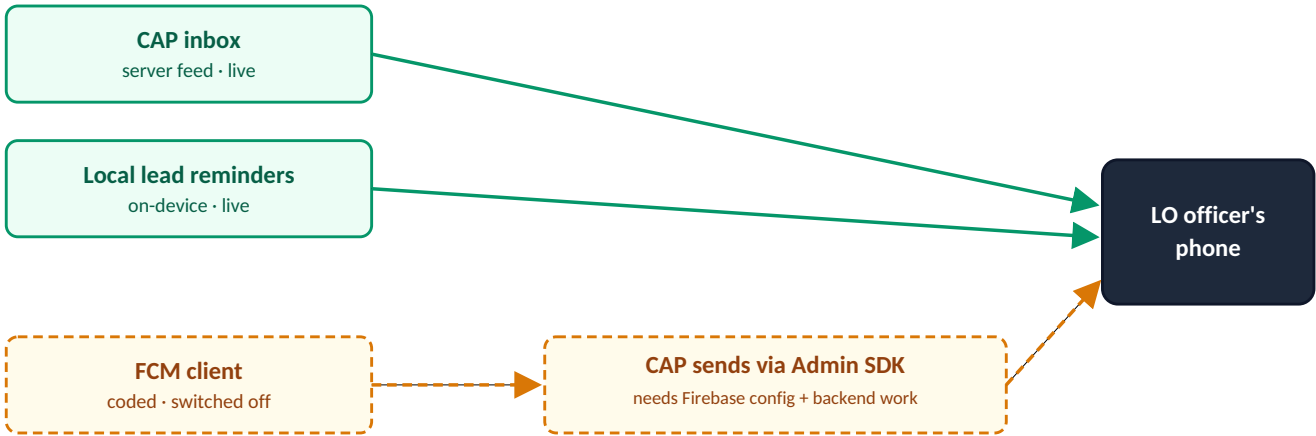
Four channels carry information to a Liaison Officer today, or will once remote push is switched on. Three are live and working against the real CAP backend. The fourth — FCM remote push — is fully coded but deliberately off, waiting on a Firebase project and two small pieces of backend work.

Channel	What it does	Status
CAP inbox	Portal notification feed, opened from the bell icon	Live
Alerts tab	On-device log of what the app itself has done	Live
Local lead-time reminders	OS notifications scheduled from task data already on the phone	Live
FCM remote push	CAP pushes straight to a locked or backgrounded phone	Coded, off

The first three don't touch Firebase at all — they work today with no further setup. What's blocking the fourth isn't anything in the Flutter app; it's two things outside it: a real Firebase project, and two endpoints' worth of work on CAP (section 6).

2. Architecture

Three channels reach the phone in different ways today; a fourth is wired but not yet switched on.



Dashed boxes and arrows mark the part of the flow that is coded but not yet live.

The Alerts tab is deliberately left out of this diagram — it's a fourth, related but separate on-device log rather than a push channel (section 3.2).

3. What's live today

All three channels below work right now against the live CAP backend. None of them need a Firebase project or any extra configuration.

3.1 CAP inbox

The bell icon on the AppBar opens the inbox screen, authenticated with the same Bearer JWT as the rest of the portal.

Method	Path	Behaviour
GET	/app/notifications/mine	Latest items — CAP returns up to about 50
POST	/app/notifications/mine/{id}/read	Mark one read
POST	/app/notifications/mine/read-all	Mark all read
GET	/app/notifications/mine/unread-count	Badge count

Each item carries `id`, `title`, `message`, `kind`, `link`, `isRead`, `createdAt`, `readAt`. The unread badge loads when the shell opens and again when the officer leaves the inbox — it doesn't poll and doesn't use a websocket, so it can sit stale while they're elsewhere in the app (see the badge-freshness note in section 6).

ROUTING RULE

A `link` containing `task` switches the app to the Tasks tab; one containing `delegate` switches to the Delegates tab. The same rule fires on an FCM tap, so both channels will route identically once push is live.

3.2 Alerts tab

The bottom-nav **Alerts** tab (`LoNotificationsScreen`) is a different thing from the CAP inbox — it's a local log of what the app itself has done: task status changes, travel and movement updates, profile submission, uploads, and issue submission or queueing. Those rows come from the local portal cache, not from CAP.

The same screen also carries:

- A lead-time picker — 15, 30, 60, or 120 minutes, default 60, stored in `SharedPreferences` as `lo_alert_lead_minutes`
- A shortcut into the CAP inbox
- A shortcut to issue reports, including how many are still unsynced

3.3 Local OS reminders

`PushNotificationService.scheduleTaskLeadReminders` runs after the portal loads tasks, and again whenever the lead time changes.

- **Package:** `flutter_local_notifications`
- **Time zone:** Asia/Kolkata, falling back to UTC if that zone data is missing
- **Channel:** `lo_task_reminders` ("Task reminders")
- **Scope:** tasks that are not completed or cancelled and that have a date
- **Timing:** scheduled date + time, minus the lead minutes — if that moment has already passed but the task itself is still upcoming, the reminder fires immediately
- **Tap payload:** `task:<id>`, opening the Tasks tab once the shell is up

PERMISSIONS

The Android manifest already declares `POST_NOTIFICATIONS`, `RECEIVE_BOOT_COMPLETED`, `VIBRATE`, and `SCHEDULE_EXACT_ALARM`, plus the scheduled-notification receivers — but scheduling itself uses inexact-while-idle mode, so despite that last permission being declared, reminders aren't exact alarms.

4. FCM remote push — coded, off by default

The client-side code for remote push is complete and already sitting in the app. It simply doesn't run unless a build opts in *and* a real Firebase Android config was present when that build was made.

What happens when it's switched on

Once `ENABLE_FCM=true` and `Firebase.initializeApp()` succeeds:

1. The app requests notification permission and reads the FCM registration token.
2. It `POST`s `/api/devices/register` with `{ email, fcmToken, platform }` (`android` or `ios`).
3. It repeats that `POST` on token refresh, for as long as a valid session exists.
4. A foreground message displays on channel `lo_push` ("CAP alerts"), with title and body taken from the notification block, or from data keys `title` / `body` if that's absent.
5. A tap reads the data key `link` and routes the same way as the CAP inbox — `task` to Tasks, `delegate` to Delegates.
6. If CAP returns a 404 (expected today) or any other error on that registration call, the failure is only logged — the inbox and local reminders carry on

regardless.

`ENABLE_FCM` defaults to false. The Google Services Gradle plugin in `android/app/build.gradle.kts` is only applied when `android/app/google-services.json` actually exists, so a release build made without that file still succeeds — it just ships with push silently off. There's no `google-services.json` in the repository, and there shouldn't be a placeholder one committed.

WORTH NOTING

Logging out drops the FCM token from memory on the device only — CAP isn't told to invalidate it. Worth keeping in mind once server-side sending is live.

5. Sending from CAP to the phone — architecture and cost

The pattern for a web backend pushing to a Flutter app is a well-worn one, and it's exactly what CAP is meant to do here:

1. **Token registration** — already built. The Flutter app requests a token from Firebase on launch and sends it to CAP via `POST /api/devices/register`.
2. **Trigger** — an event happens on CAP: a task is assigned or updated, a schedule changes, or a B2B meeting is requested for that officer's delegates.
3. **API request** — CAP's backend uses the Firebase Admin SDK to send a payload straight to Firebase's servers.
4. **Delivery** — Firebase pushes it to the officer's Android or iOS device, even if the app isn't open.

COST

Delivery itself doesn't cost anything. Firebase Cloud Messaging carries no per-message fee on the free Spark plan, with no cap on message volume or device count — as long as the request goes directly from a backend to Firebase via the Admin SDK or the HTTP v1 API, which is exactly this design.

DOESN'T APPLY HERE

The one caveat that shows up in general FCM guidance is about using Cloud Firestore or Cloud Functions to auto-trigger notifications — those carry their own free-tier limits (for example, 20,000 Firestore writes a day) and start charging past that. This project doesn't route through either: CAP already holds the tasks and the in-app feed, and sends directly through the Admin SDK. The whole flow sits inside the free, uncapped path.

Limit	Figure	What it means here
Total messages	Unlimited — billions a month, no charge	No ceiling on send volume
Registered devices	Unlimited	No cap on how many officers can register a token
Throughput	600,000 msgs/min per project (HTTP v1); 429 past that until the next minute	Far beyond anything an LO Committee roster will produce
Message payload	4 KB total	The suggested title / body / link payload sits comfortably inside this
Upstream (device→server)	10,000 requests/min per project	Not relevant — this app doesn't send device-to-server FCM messages
Device group size	20 members max	Not used yet — one token per officer, no grouping
Topic subscriptions	Unlimited per topic	Not used — sends are targeted to one officer, not topic-based
Concurrent topic broadcasts	1,000 per project	Not used

General Firebase Cloud Messaging platform limits, current as of this document's preparation — worth a quick check against Firebase's own documentation if this section is relied on much later.

6. Still needed before FCM goes live

Four pieces of outside work, plus two smaller follow-ups.

Firebase project (mobile config)

1. Create a Firebase project and add an Android app with application ID `com.bel.liaison_officer`.
2. Place the real `google-services.json` in `android/app/`.
3. Build and run with `ENABLE_FCM=true` alongside the usual live-CAP defines:

```
flutter run --release `
  --dart-define=USE MOCK_API=false `
  --dart-define=API_BASE_URL=http://35.244.48.209:8080 `
  --dart-define=ENABLE_FCM=true
```

Without the JSON file, `Firebase.initializeApp()` fails and the service stays off. Without `ENABLE_FCM=true`, the service never calls Firebase at all — both have to be satisfied together.

CAP must store the token

`POST /api/devices/register` isn't in the live Swagger — the app already calls it and quietly ignores the 404. CAP needs to accept:

```
{ "email": "lo@example.com", "fcmToken": "<device token>", "platform": "android" }
```

Store it against that Liaison Officer, and replace the previous token for the same device when FCM rotates it — the app POSTs again on every refresh.

CAP must send the push

On a task being assigned or updated, a schedule change, or a B2B meeting request for that officer's delegates, CAP looks up the stored token and sends through the Firebase Admin SDK (FCM HTTP v1). Suggested payload, kept well under the 4 KB limit:

Key	Example
title	Task assigned
body	Airport pickup · Delegate name
link	task:<taskId> or delegate:<assignmentId>

SECURITY

The Admin SDK private key stays on the CAP server only — never in the Flutter app, in git, or inside the APK.

iOS

This tree is Android-first today. Missing before an iPhone can receive the same pushes: `GoogleService-Info.plist`, the Push Notifications capability, the Remote notifications background mode, and an APNs key uploaded to the Firebase project.

Two smaller follow-ups

- ☐ **Badge freshness** — the unread count only refreshes on shell start and on returning from the inbox; it doesn't update while the officer stays on Delegates or Tasks. A later pass could poll `unread-count` or refresh when a push arrives.
- ☐ **Help copy** — `help_support_screen.dart` still describes FCM as an optional follow-up and says local reminders cover lead times. That's still accurate for builds without `ENABLE_FCM`, but it's worth revisiting once push actually ships.

Appendix A — Notification API reference

Base URL `http://35.244.48.209:8080` · auth header `Authorization: Bearer <accessToken>` · every response wrapped in `{ success, message, data, errorCode, timestamp }`.

Method	Path	Notes	Swagger
GET	<code>/app/notifications/mine</code>	In-app feed	Confirmed
POST	<code>/app/notifications/mine/{id}/read</code>	Mark one read	Confirmed
POST	<code>/app/notifications/mine/read-all</code>	Mark all read	Confirmed
GET	<code>/app/notifications/mine/unread-count</code>	Badge count	Confirmed
POST	<code>/api/devices/register</code>	<code>{ email, fcmToken, platform }</code> ; sent after OTP login and on token refresh when <code>ENABLE_FCM=true</code>	Not in Swagger — 404 logged & ignored

CAP — not the Flutter app — looks up the stored token and sends through the Firebase Admin SDK. The private key never belongs in the app.

Appendix B — Source map

Piece	File
Notifications repository (CAP inbox)	<code>lib/features/liaison_officer/data/repository/dio_notifications_repository.dart</code>
Notifications contract	<code>lib/features/liaison_officer/domain/notifications_repository.dart</code>
Inbox screen	<code>lib/features/liaison_officer/presentation/screens/notifications_inbox_screen.dart</code>
Alerts tab screen	<code>lib/features/liaison_officer/presentation/screens/lo/lo_notifications_screen.dart</code>
Local reminder scheduling	<code>lib/core/services/push_notification_service.dart</code>
FCM token registration	<code>lib/core/services/fcm_device_registrar.dart</code>
Help screen (copy needs a refresh)	<code>lib/features/liaison_officer/presentation/screens/help_support_screen.dart</code>
Firebase Gradle plugin gate	<code>android/app/build.gradle.kts</code>

Branch `feature/lo-mobile-production` · requirement #11 / LO.9.4 · fuller store, permissions, and architecture notes live in `store-launch-and-notifications-guide.md`.

This document consolidates the notifications implementation-status notes, the CAP notification API reference, and current Firebase Cloud Messaging platform limits into one reference.